IMD3002 – 3D COMPUTER GRAPHICS

Prof. Chris Joslin

L23 – Shadows & Effects

Overview

- This lecture will introduce non-ray-traced shadows and the fog effect

Effects

- Introduction
  - When we are discussing rendering there are few visual elements that would be difficult to obtain using polygons or surfaces
  - Therefore, we use specific “effects” to achieve a rendering for these specific items which are a very specific to that effect (rather than a specific type of rendering)

Effects

- Introduction
  - In this group of effects are the following:
    - Fast Shadows
    - Fog
    - Fire
    - Lightning
    - Explosions, etc.
  - This is not an exhaustive list, but it does cover some of the most important elements

Shadows

- Introduction
  - Shadows are important elements in creating realistic images and in providing the user with visual cues about an object's placement (and the light too)
  - There are many ways to create shadows, and we shall cover some of the more pertinent here

Shadows

- Introduction
  - Here is a list of most of the methods for creating shadows:
    - Planar Shadows
      - Projection Shadows
      - Soft Shadows
    - Shadow Volumes
    - Shadow Maps

Shadows

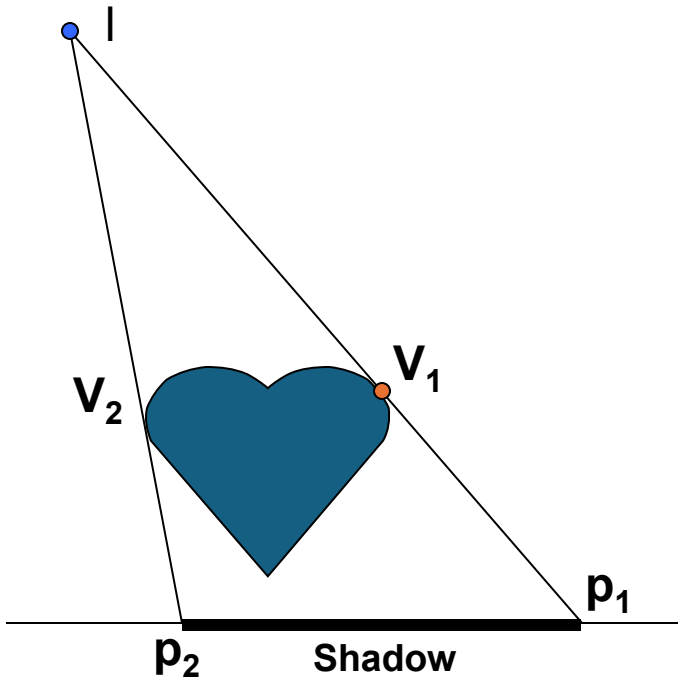
- Planar Shadows
  - A simple case of shadowing occurs when objects cast shadows on planar surfaces; two kinds of algorithms for planar shadows are presented here:
    - Projection Shadows
    - Soft Shadows

Shadows

- Planar Shadows : Projection Shadows
  - Using Projection Shadows, a 3D object is rendered a second time in order to create a shadow
  - A matrix can be derived that projects the vertices of an object onto a plane
  - Consider a light source located a position l, and the vertex to be projected at position v, and the projected vertex at position p

Shadows

- Planar Shadows : Projection Shadows
  - Here is a diagram showing that situation



Shadows

- Planar Shadows : Projection Shadows
  - In this situation a light source, located at l, casts a shadow onto the plane where y=0
  - The vertex **v** is projected onto the plane, and the projected point is **p**
  - The similar triangles are used for the derivation of the projection matrix

Shadows

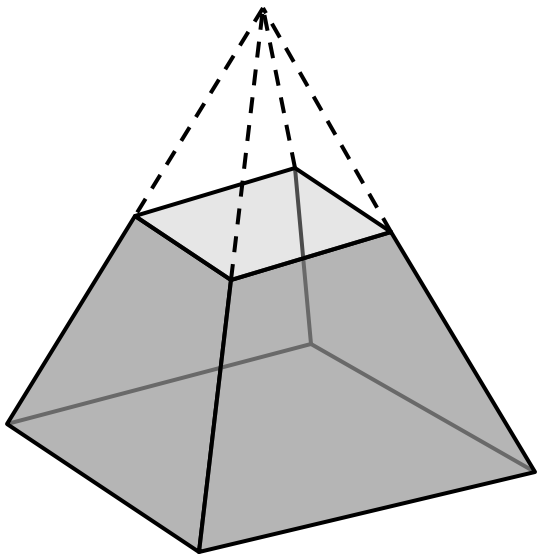
- Planar Shadows : Soft Shadows
  - Projective shadows can also be made soft, by using a variety of techniques
  - Soft shadows appear whenever a light source has an area
  - One way to approximate the effect of an area light is to sample it by using a point lights placed on its surface
  - For each point light an image is rendered and added to the buffer
  - The average of these images is then an image with soft shadows

Shadows

- Shadow Volumes
  - We can cast shadows onto arbitrary objects by using a stencil buffer; this technique is sometimes called "Volumetric Shadows"
  - Imagine a point and a triangle; by extending lines from the point through the vertices of the triangle to infinity we obtain an infinite pyramid
  - The part under the triangle, i.e. the opposite side to the point, is a truncated infinite pyramid
  - The upper part is simply a pyramid

Shadows

- Shadow Volumes
  - Assume the point is a light source and therefore any part of an object which is inside the volume of the truncated pyramid is in shadow; this volume is called the "shadow volume"



Shadows

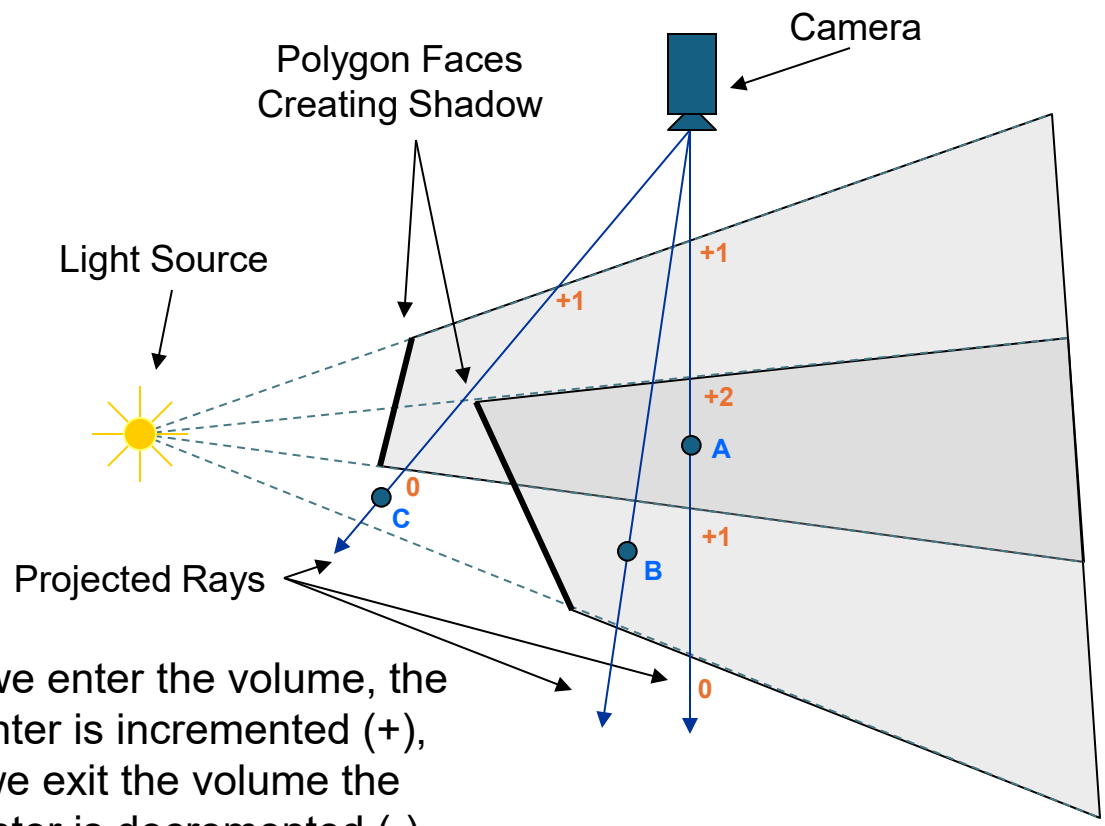
- Shadow Volumes
  - Now, for a scene we can follow a projected ray through a pixel until that ray hits the object to be displayed on the screen
  - While the ray is on its way to the object, we increment a counter each time it crosses a face of the shadow volume that is front facing (i.e. facing toward the camera)
  - Therefore, the counter is incremented each time the ray goes into shadow

Shadows

- Shadow Volumes
  - In the same manner, we decrement the same counter each time the ray crosses a back-facing side of the truncated pyramid
  - The ray is then going out of a shadow; the ray hits the object that is to be displayed at that pixel – if the counter is greater than zero, then that pixel is in shadow (otherwise it is not)
  - This principle works when there is more than one polygon that casts shadows

Shadows

- Shadow Volumes
  - **Example:** shadow counting



As we enter the volume, the counter is incremented (+), as we exit the volume the counter is decremented (-)

We can see here two polygons creating shadow volumes (from a side view)

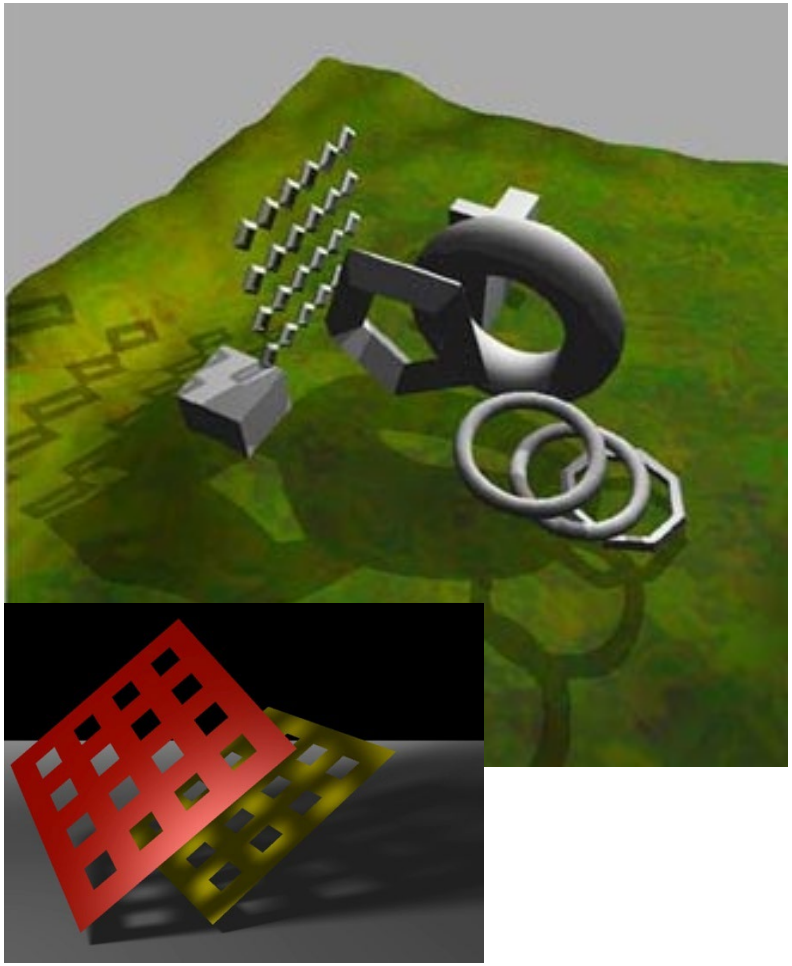
Three intersection points (e.g. on objects) are examined:

- Point A: +, + (2 = shadow)
- Point B: +, +, - (1 = shadow)
- Point C: +, - (0 = no shadow)

Any value above 0 for the counter is going to make a point in shadow

Shadows

- Shadow Volumes



Shadows

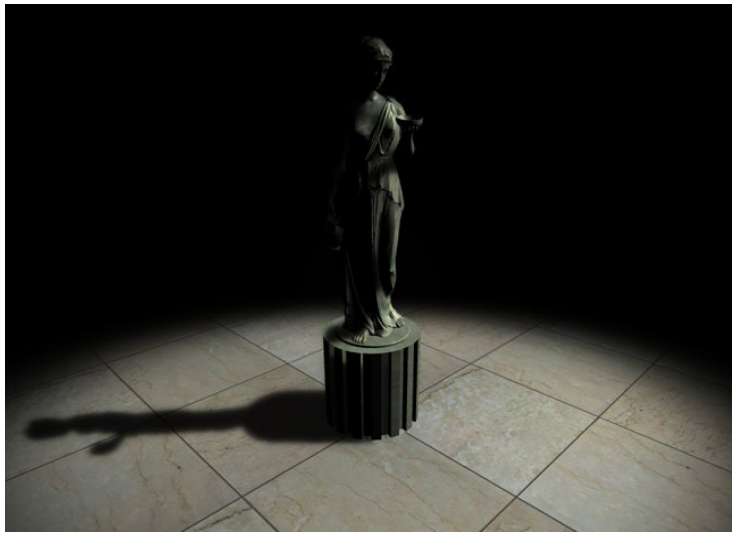
- Shadow Maps
  - Shadow maps work using a Z-buffer based renderer
  - The main idea is to render the scene, using the Z-buffer algorithm, from the position of the light source that is to cast the shadows we are concerned with
  - For each pixel in the z-buffer, the z-depth now contains the distance to the object closest to the light source

Shadows

- Shadow Maps
  - We call the entire contents of the Z-buffer the "Shadow Map", also sometimes called the "Shadow Depth Map", or the "Shadow Buffer"
  - To use the shadow map, the scene is rendered from a second time, but this time from the perspective of the viewer
  - As each drawing primitive is being rendered, its location is compared to the shadow map; if a rendered point is farther away from the light source that the value in the shadow map, then that point is in shadow

Shadows

- Shadow Maps
  - If the point is the same as the point in the shadow map, then it is lit (or not in shadow)
  - Here we see the two rendered conditions, first from the light's perspective, then from the eye's perspective

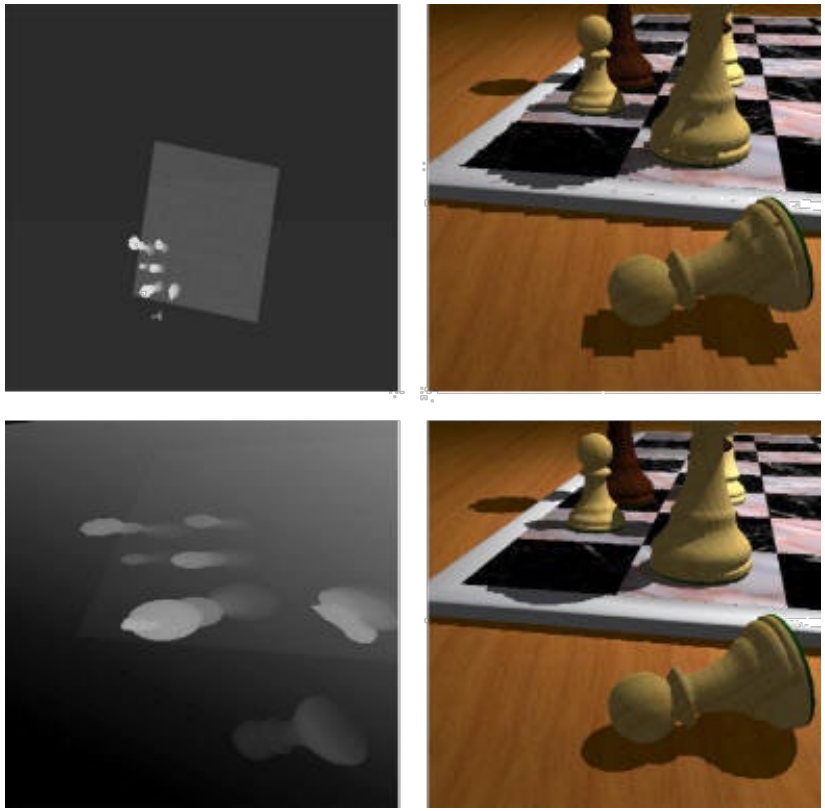


Shadows

- Shadow Maps
  - It is important to remember that whilst we have to render two frames, we only have to do this each time if the object (we are creating the shadow for) or the light is actually moving
  - Therefore, even if the camera is moving, there is no need to re-render for the light's position as it will remain the same
  - It is also important to remember that even the properties of the light can change (and the material of the object) and the shadow map will remain the same

Shadows

- Shadow Maps
  - **Example:** shadow mapped object (the images on the left show the actual shadow map produced)



Shadows

- Shadow Maps
  - **Example:** shadow mapping techniques

Fog

- Introduction
  - Fog effects are an important part of realistic computer graphics
  - **Example:** flight simulators often use fog effects to simulate adverse weather conditions - this allows air crews to be safely familiarized with difficult operational scenarios, such as landing on fog-obscured runways
  - To be realistic, fog effects must closely model the appearance and behavior of real fog conditions

Fog

- Introduction
  - This means that fog effects must be capable of modelling patchy or otherwise non-uniform fog or haze
  - Realistic fog effects must also be capable of animation; this allows fog to swirl or move in a manner that mimics natural fog
  - Several techniques have been developed that can be used within the stages of a graphics pipeline to create fog effects; these include: colour blending, texture mapping, and volumetric rendering

Fog

- Colour Blending
  - For colour blending, a fog colour is selected or defined (usually a shade of grey)
  - During the “fogging process”, the graphics pipeline blends the fog colour into each of the pixels within the image being fogged
  - The graphics pipeline determines the amount of fog colour to add to the image’s pixels by calculating a weighting factor for each pixel

Fog

- Colour Blending
  - Each pixel's weighting factor is a linear or exponential function of the distance between the pixel and the eye-point
  - The graphics pipeline blends the predefined fog colour into the each pixel in accordance with the pixel's weighting factor

Fog

- Texture Mapping
  - For texture mapping, a series of texture maps are generated
  - Each texture map is configured to make objects appear as if they are being viewed through a specific fog depth
  - Thus, a first texture map might make objects appear as if they were obscured by one meter of fog and a second texture map might make objects appear as if they were obscured by ten meters of fog, etc.

Fog

- Texture Mapping
  - In some cases, the series of textures are encoded within a 3D texture map
  - During the fogging process, the graphics pipeline selects the appropriate texture map for each primitive within the image being fogged
  - Selection of texture maps is based on the distance between the primitives and the eye
  - The pipeline then textures each primitive with the appropriate texture map

Fog

- Volumetric Rendering
  - With volumetric rendering, a 3D volume is generated to model the desired fog effect
  - During the fogging process, the graphics pipeline applies the 3D volume to the space between the primitives included in an image and the eye-point
  - Typically, this is accomplished through the use of a volumetric rendering technique such as ray-casting, voxel rendering, or 3D texture-slice composition

Fog

- Technique Comparison
  - Colour blending, texture mapping and volumetric rendering are all effective techniques for rendering uniform fog effects
  - However, these techniques are subject to disadvantages that can make them undesirable
  - For example, when using colour blending to create fog effects, the amount of fog colour is controlled strictly by pixel distance
  - This means that fog effects produced by these techniques tend to be uniform (non-patchy) and constant over time (inanimate)

Fog

- Technique Comparison
  - Texture mapping and volumetric rendering each allow for the creation of non-uniform (patchy) fog effects
  - For this reason both of these techniques offer a higher degree of realism than is possible for colour blending
  - Unfortunately, neither texture mapping nor volumetric rendering are able to provide animated fog effects

Fog

- Technique Comparison
  - Both of these techniques are also limited in other fashions
  - Texture mapping, fails to provide a mechanism that allows the appearance of a fog effect to change as the eye-point moves in relation to an image
  - Volumetric rendering overcomes this difficulty but generally requires expensive hardware support within the GPU